

```
//SIT 55 : ASSIGNMENT 2 : STACK
/*2.Implement stack as an abstract data type using singly linked list and
use this ADT for conversion of
infix expression to postfix, prefix and evaluation of postfix and prefix
expression.*/
```

```
#include<iostream>
using namespace std;
char Postfix[30], Prefix[30];
struct node
{
char data;
node *next;
};
//=====
class stk
{
public:
int newtop=-1;
int arr1[30];
int newempty()
{
if(newtop==-1)
return 1;
else
return 0;
}
int newfull()
{
if(newtop==30)
return 1;
else
return 0;
}
void newpush(int data)
{
if(!newfull())
{
newtop++;
arr1[newtop]=data;
}
else
{
cout<<"\n Stack is full!!!";
}
}
int newpop()
{
if(!newempty())
{
newtop--;
return arr1[newtop+1];
}
else
{
cout<<"\n Stack is empty!!!";
}
return 0;
}
};
```

```
//=====
class Stack : public stk
{
public:
node *top;
Stack()
{
top=NULL;
}
int empty()
{
if(top==NULL)
{
return 1;
}
else
{
return 0;
}
}
void push(char c)
{
node *newNode;
newNode=new node;
newNode->data=c;
newNode->next=top;
top=newNode;
}
char pop()
{
if(!empty())
{
node *temp;
temp=top;
top=top->next;
return temp->data;;
}
return 0;
}
//-----
int check_Operator_or_Operand(char ch)
{
if(ch=='+' || ch=='-' || ch=='*' || ch=='/' || ch=='(' || ch==')')
{
return 1;
}
else if(ch>='A' && ch<='Z' || ch>='a' && ch<='z')
{
return 0;
}
else
{
cout<<"\nWrong Expression !!!!!!!!!";
}
return 0;
}
int precedance(char ch)
{
if(ch=='*' || ch=='/')
{

```

```

return 2;
}
else if(ch=='+' || ch=='-')
{
return 1;
}
return 0;
}
// -----
void Infix_To_Postfix(char Infix[])
{
int i=0;
int j=0;
while(Infix[i]!='\0')
{
if(check_Operator_or_Operand(Infix[i])==0)
{
Postfix[j]=Infix[i];
j++;
}
else
{
if(top!=NULL && top->data!='(' && top->data!=')' &&
precedance(top->data)>=precedance(Infix[i]))
{
Postfix[j]=pop();
j++;
push(Infix[i]);
}
else
{
if(Infix[i]==')')
{
while(top->data!='(')
{
Postfix[j]=pop();
j++;
}
pop();
}
else
{
push(Infix[i]);
}
}
}
i++;
}
while(top!=NULL)
{
Postfix[j]=top->data;
pop();
j++;
}
}
//-----
void Infix_To_Prefix(char Infix[])
{
int i=0;
int j=0;

```

```

while(Infix[i]!='\0')
{
if(check_Operator_or_Operand(Infix[i])==0)
{
Prefix[j]=Infix[i];
j++;
}
else
{
if(top!=NULL && top->data!='(' && top->data!=')' &&
precedance(top->data)>precedance(Infix[i]))
{
Prefix[j]=pop();
j++;
push(Infix[i]);
}
else
{
if(Infix[i]==')')
{
while(top->data!='(')
{
Prefix[j]=pop();
j++;
}
pop();
}
else
{
push(Infix[i]);
}
}
}
i++;
}
while(top!=NULL)
{
Prefix[j]=top->data;
pop();
j++;
}
// -----
void evaluatePostfix()
{
int data,r1,r2,ans;
for(int i=0;Postfix[i]!='\0'; i++)
{
char ch=Postfix[i];
if(check_Operator_or_Operand(ch)==0)
{
cout<<"\nEnter data for "<<ch<<" : ";
cin>>data;
newpush(data);
}
else
{
r2=newpop();
r1=newpop();
ans=evaluate(ch,r1,r2);
}
}
}

```

```

newpush(ans);
}
}
int val=newpop();
cout<<"\nValue of expression is : "<<val;
}
// -----
void evaluatePrefix()
{
int data,r1,r2,ans;
for(int i=0;Prefix[i]!='\0'; i++)
{
char ch=Prefix[i];
if(check_Operator_or_Operand(ch)==0)
{
cout<<"\nEnter data for "<<ch<<" : ";
cin>>data;
newpush(data);
}
else
{
r2=newpop();
r1=newpop();
ans=evaluate(ch,r1,r2);
newpush(ans);
}
}
int val=newpop();
cout<<"\nValue of expression is : "<<val;
}
// -----
int evaluate(char x,int op1,int op2)
{
if(x=='+')
return(op1+op2);
if(x=='-')
return(op1-op2);
if(x=='*')
return(op1*op2);
if(x=='/')
return(op1/op2);
if(x=='%')
return(op1%op2);
return 0;
};
int main()
{
Stack obj;
char Infix[30];
int ch;
bool a=true;
do{
cout<<"\n1. Infix to Postfix \n2. Infix to Prefix \n3. Evaluate Postfix
\n4. Evaluate Prefix \n5. Exit \n";
cout<<"Enter your choice : ";
cin>>ch;
if(ch==1)
{
cout<<"Enter your Infix Expression : ";

```

```

cin>>Infix;
obj.Infix_To_Postfix(Infix);
cout<<"\nPostfix Expression : "<<Postfix;
}
else if(ch==2)
{
cout<<"Enter your Infix Expression : ";
cin>>Infix;
//-----
char arr[30],temp;
int j,len=0;
for(int i=0;Infix[i]!='\0';i++)
{
len++;
}
j=len-1;
for(int i=0;Infix[i]!='\0';i++)
{
arr[i]=Infix[j];
j--;
}
//-----
for(int i=0;arr[i]!='\0';i++)
{
if(arr[i]==')')
{
arr[i]='(';
}
else if(arr[i]=='(')
{
arr[i]=')';
}
}
//-----
obj.Infix_To_Prefix(arr);
// -----
char arr1[30];
j=0;
len=0;
for(int i=0;Prefix[i]!='\0';i++)
{
len++;
}
j=len-1;
for(int i=0;Prefix[i]!='\0';i++)
{
arr1[i]=Prefix[j];
j--;
}
cout<<"\nPrefix Expression : "<<arr1;
}
else if(ch==3)
{
obj.evaluatePostfix();
}
else if(ch==4)
{
obj.evaluatePrefix();
}
else

```

```

{
return 0;
}
cout<<"\nDo you want to Continue (1/0) :";
cin>>a;
}while(a);
return 0;
}

```

/*OUTPUT:

1. Infix to Postfix
2. Infix to Prefix
3. Evaluate Postfix
4. Evaluate Prefix
5. Exit

Enter your choice : 1

Enter your Infix Expression : a+b*c

Postfix Expression : abc*+

Do you want to Continue (1/0) :1

1. Infix to Postfix
2. Infix to Prefix
3. Evaluate Postfix
4. Evaluate Prefix
5. Exit

Enter your choice : 2

Enter your Infix Expression : a*b+c

Prefix Expression : +*abc

Do you want to Continue (1/0) :1

1. Infix to Postfix
2. Infix to Prefix
3. Evaluate Postfix
4. Evaluate Prefix
5. Exit

Enter your choice : 3

Enter data for a : 6

Enter data for b : 8

Enter data for c : 2

Value of expression is : 22

Do you want to Continue (1/0) :1

1. Infix to Postfix
2. Infix to Prefix
3. Evaluate Postfix
4. Evaluate Prefix
5. Exit

Enter your choice : 4

Enter data for c : 6

Enter data for b : 8

Enter data for a : 2

Value of expression is : 22

Do you want to Continue (1/0) :1

1. Infix to Postfix

2. Infix to Prefix
3. Evaluate Postfix
4. Evaluate Prefix
5. Exit

Enter your choice : 5*/